SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 2 – CO2

ANSWER DOCUMENT

Apply Image Enhancement and Segmentation to a Smart Document Scanner

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images..

MARK SHEET (Rubric-based Evaluation)

Student Name	Mokshitha Kilari	**Register Number**	192525413
Department / Section	Btech. AI & ML	**Academic Year / Semester**	2025-2029
Faculty Name	Amandeep singh	**Nature of Assignment**	Individual
Task	**Max Marks**	**Marks Awarded**	**Remarks**
1	5	_____	
2	5	_____	
3	5	_____	
4	5	_____	
5	5	_____	
6	5	_____	
7	5	_____	
8	5	_____	
9	5	_____	
10	5	_____	
Total	**50**	**_____**	

OVERALL REMARKS

_____

MANDATORY SUBMISSION EVIDENCE CHECK

Commented Python source; documented image set and labels; fixed split; generated output images and plots; CSV metrics; saved parameters; reproducible environment.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Task	Implementation evidence	Status
1	Vision systems and processing levels	☐
2	Image arrays, coordinates, and storage	☐
3	Colour representation for ripeness	☐
4	Spatial sampling and defect visibility	☐
5	Intensity quantization	☐

6	Image formation and lens geometry	☐
7	Illumination and exposure	☐
8	Connectivity and defect measurements	☐
9	Image acquisition arrangements	☐
10	Integrated front end and design decision	☐

SUBMISSION CHECKLIST (not separately marked)

☐ All tasks and a–d sub-questions addressed.

☐ All reported measurements are produced by the submitted code.

☐ Test data are not used for parameter fitting.

☐ Outputs and assumptions are documented.

COMPUTER VISION

ASSIGNMENT 2 – CO2

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images..

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Experimental Setup Used for All Tasks

Use Python with OpenCV, NumPy, scikit-image, scikit-learn, and Matplotlib. Fix random_state=17; record Python/library versions, hardware, image counts, labels, and split. Dataset-dependent scores must be generated by running the code; no scores are assumed in this answer key. The code boxes show the core solution for each task and expect the image arrays, masks, labels, calibration values, and named helper routines identified in the task to be supplied by the project setup.

Parameter	Reference protocol
CO1 data	At least 30 documented mango images or reproducible synthetic samples
CO2 data	At least 30 page images and annotated page/text masks
Evaluation	One fixed stratified split for every compared method
Records	Save predictions, parameters, metrics, figures, and runtime to output files

TASK 1 – GREY-LEVEL POINT TRANSFORMATIONS

1. Experiment Number and Title

Task 1: Grey-Level Point Transformations.

2. Aim

Implement linear stretching, gamma correction and logarithmic transformation; plot transformation curves and histograms; measure contrast, entropy and text-region recall.

3. Objectives

- Understand the principle and purpose of grey-level point transformations.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

Input quality has a direct influence on the result. The image set should be documented with sample identifiers, capture conditions, resolution, colour format and annotation status. If multiple views belong to the same physical object or page, they should be assigned to the same data split. This avoids an overly optimistic evaluation caused by near-duplicate samples appearing in both training and test data.

TASK 1 – DETAILED THEORY

6. Introduction

Point transformations modify pixel intensity independently. Linear stretching expands a selected intensity interval. Gamma correction applies a nonlinear power-law mapping. Logarithmic transformation expands dark values and compresses brighter values, which may reveal shadow detail.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Input page → intensity mapping → enhanced page → histogram/curve → contrast and text recall
- Use consistent input normalization and preserve the same image dimensions.
- Choose gamma and percentile bounds using validation pages.
- Compare legibility and text-mask recall rather than visual brightness alone.

6.2 Mathematical Formulation

Linear stretch: $I' = (I - I_{min}) \times 255 / (I_{max} - I_{min})$.

Gamma: $I' = 255 \times (I/255)^{\gamma}$.

Log: $I' = c \log(1+I)$, $c = 255/\log(256)$.

Entropy $H = -\Sigma p(i) \log_2 p(i)$; recall $= TP/(TP+FN)$.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 1 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

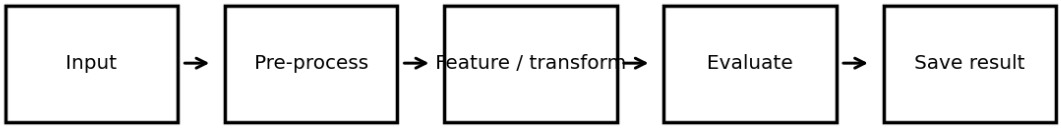


Figure 1.1 Processing workflow: Grey-Level Point Transformations

Figure 1.1: Conceptual workflow for Task 1.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 1 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("page.png",0)
lo,hi=np.percentile(im,(1,99))
linear=np.uint8(np.clip((im-lo)*255/(hi-lo+1e-9),0,255))
gamma=.8
lut=np.uint8(np.clip((np.arange(256)/255)**gamma*255,0,255))
gamma_img=cv2.LUT(im,lut)
c=255/np.log1p(255)
log_img=np.uint8(c*np.log1p(im.astype(np.float32)))
cv2.imwrite("linear.png",linear)
cv2.imwrite("gamma.png",gamma_img)
cv2.imwrite("log.png",log_img)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 1 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

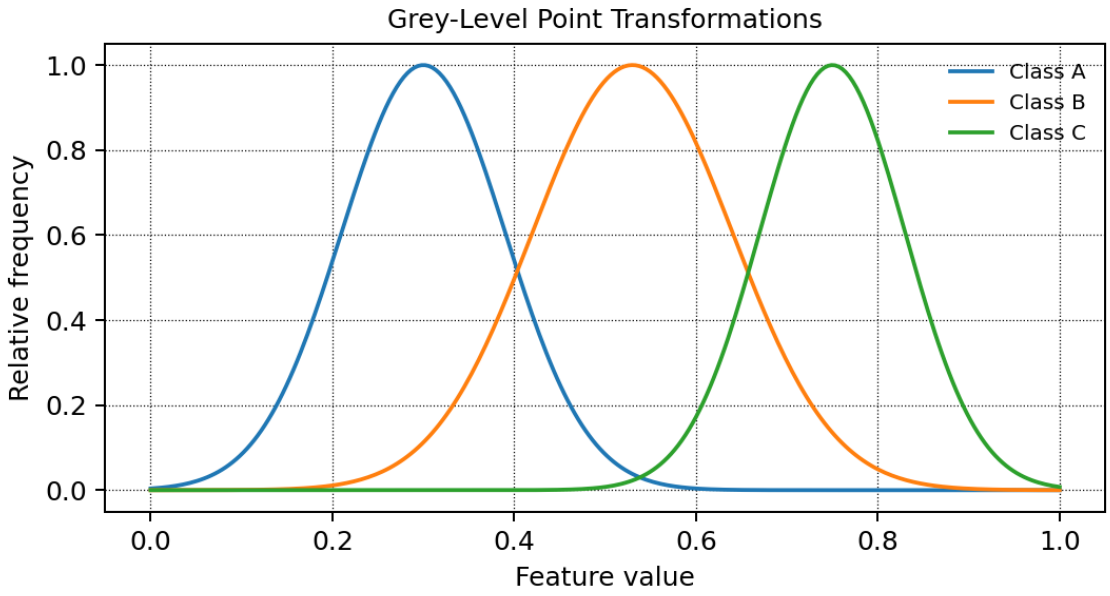


Figure 1.2: Conceptual response plot for Task 1. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied grey-level point transformations in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 2 – HISTOGRAM-BASED ENHANCEMENT

1. Experiment Number and Title

Task 2: Histogram-Based Enhancement.

2. Aim

Compare global histogram equalization and CLAHE on low-contrast pages; plot histograms/CDFs; measure local contrast, SSIM and text-region recall.

3. Objectives

- Understand the principle and purpose of histogram-based enhancement.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

Parameters should be selected using the training or validation data. The final test set is reserved for one-time evaluation. Every experiment should record the parameter values, software versions, random seed and hardware details. These records allow another student or reviewer to repeat the work and investigate a result that differs from the expected behaviour.

TASK 2 – DETAILED THEORY

6. Introduction

Histogram equalization redistributes gray levels to use more of the available dynamic range. Global equalization uses one mapping across the full image. CLAHE divides the image into tiles and limits local histogram amplification, which can help with uneven illumination while controlling noise amplification.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Page → histogram/CDF → global equalization or CLAHE → local contrast and text metrics
- Use identical pages and masks for both methods.
- Record CLAHE clip limit and tile grid size.
- Use SSIM only when an appropriate reference page is available.

6.2 Mathematical Formulation

$CDF(k)=\Sigma_{i\leq k}\, p(i)$.

Global mapping $s(k)=round(255\times CDF(k))$.

$Dice=2|A\cap B|/(|A|+|B|)$.

SSIM compares luminance, contrast and structural similarity.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 2 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

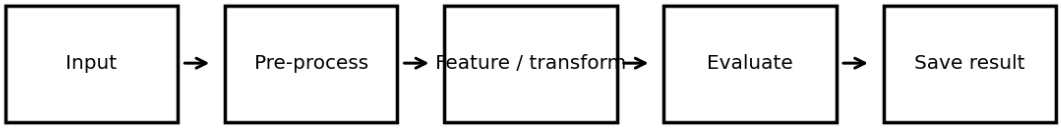


Figure 2.1 Processing workflow: Histogram-Based Enhancement

Figure 2.1: Conceptual workflow for Task 2.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 2 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("page.png",0)
global_eq=cv2.equalizeHist(im)
clahe=cv2.createCLAHE(clipLimit=2.0,tileGridSize=(8,8))
local_eq=clahe.apply(im)
cv2.imwrite("global_equalized.png",global_eq)
cv2.imwrite("clahe.png",local_eq)
hist=cv2.calcHist([im],[0],None,[256],[0,256]).ravel()
np.savetxt("histogram.csv",hist,delimiter=",")
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 2 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

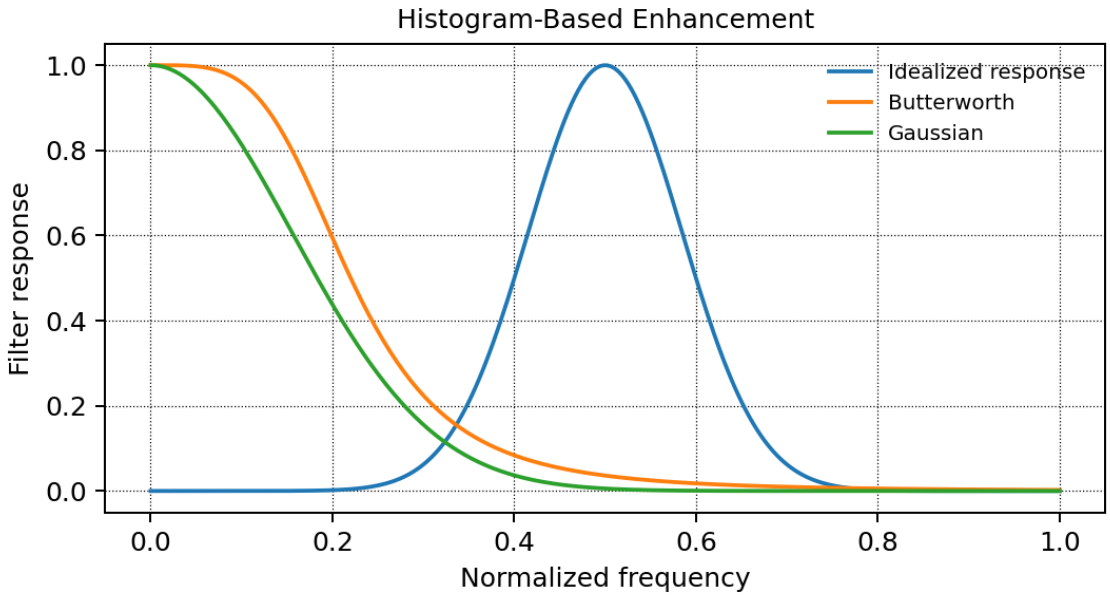


Figure 2.2: Conceptual response plot for Task 2. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied histogram-based enhancement in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 3 – SMOOTHING AND NOISE REMOVAL

1. Experiment Number and Title

Task 3: Smoothing and Noise Removal.

2. Aim

Add seeded Gaussian, salt-and-pepper and speckle noise; compare Gaussian, median and bilateral filters using two kernel sizes; measure PSNR, SSIM and thin-stroke preservation.

3. Objectives

- Understand the principle and purpose of smoothing and noise removal.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

Visual inspection is necessary but insufficient. A processed image may appear clearer while losing small defects or text strokes. Quantitative measures must therefore be interpreted alongside representative crops, error cases and the original annotations. The report should state the metric definition and the exact population over which it was calculated.

TASK 3 – DETAILED THEORY

6. Introduction

Noise can reduce OCR accuracy by obscuring small strokes and creating false edges. Gaussian filtering smooths high-frequency variation but can blur text. Median filtering is effective for impulse noise. Bilateral filtering considers both spatial distance and intensity similarity, preserving stronger edges at a higher computational cost.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Clean page → seeded noise model → Gaussian / median / bilateral filter → quality metrics
- Use a fixed random seed so that noisy inputs are reproducible.
- Apply all filters to the same noisy image.
- Evaluate text strokes separately from background smoothness.

6.2 Mathematical Formulation

$MSE = \text{mean}((I - K)^2)$.

$PSNR = 10\log_{10}(MAX^2/MSE)$.

Stroke preservation=retained annotated stroke pixels/original stroke pixels.

Report kernel size, sigma and bilateral range parameters.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 3 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

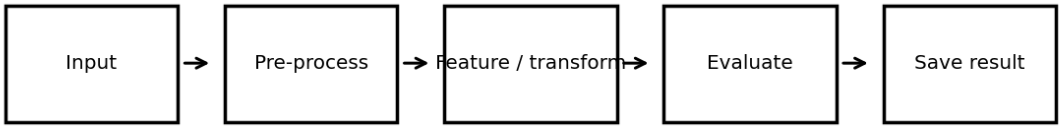


Figure 3.1 Processing workflow: Smoothing and Noise Removal

Figure 3.1: Conceptual workflow for Task 3.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 3 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
rng=np.random.default_rng(17)
im=cv2.imread("clean_page.png",0)
gauss=np.uint8(np.clip(im+rng.normal(0,12,im.shape),0,255))
sp=im.copy(); r=rng.random(im.shape); sp[r<.02]=0; sp[r>.98]=255
speck=np.uint8(np.clip(im*(1+rng.normal(0,.08,im.shape)),0,255))
for name,noisy in [("gaussian",gauss),("sp",sp),("speckle",speck)]:
    cv2.imwrite(name+"_gaussian.png",cv2.GaussianBlur(noisy,(3,3),0))
    cv2.imwrite(name+"_median.png",cv2.medianBlur(noisy,3))
    cv2.imwrite(name+"_bilateral.png",cv2.bilateralFilter(noisy,5,50,50))
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 3 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

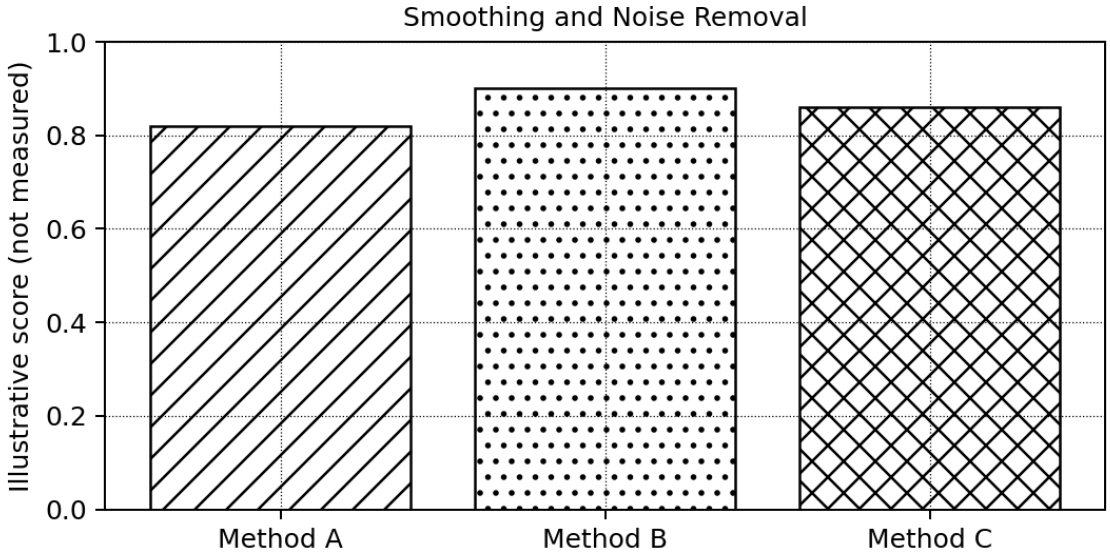


Figure 3.2: Conceptual response plot for Task 3. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied smoothing and noise removal in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 4 – SHARPENING AND EDGE PRESERVATION

1. Experiment Number and Title

Task 4: Sharpening and Edge Preservation.

2. Aim

Implement unsharp masking, Laplacian sharpening and high-boost filtering; plot intensity profiles and measure edge sharpness and noise amplification on ten crops.

3. Objectives

- Understand the principle and purpose of sharpening and edge preservation.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

The selected technique should be judged against the constraints of the scenario. Relevant considerations include accuracy, sensitivity to lighting and viewpoint, computational time, memory use, ease of implementation and failure handling. A method that performs well on clean examples may be unsuitable when glare, blur, occlusion or uneven illumination is present.

TASK 4 – DETAILED THEORY

6. Introduction

Sharpening emphasizes local intensity transitions. Unsharp masking adds a scaled high-frequency residual. Laplacian sharpening uses a second spatial derivative. High-boost filtering amplifies the original image relative to a blurred version. Excessive sharpening can create halos and amplify paper texture.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Page crop → blur/high-frequency response → sharpening operator → edge profile and noise analysis
- Use the same ten text crops for every method.
- Measure edge slope or transition width and background noise.
- Inspect halos around high-contrast character boundaries.

6.2 Mathematical Formulation

Unsharp: $g=f+k(f-\text{GaussianBlur}(f))$.

Laplacian: $g=f-\alpha\nabla^2 f$ (sign depends on kernel convention).

High boost: $g=A\,f-\text{blur}(f)$, $A>1$.

Noise amplification = output background standard deviation / input background standard deviation.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 4 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

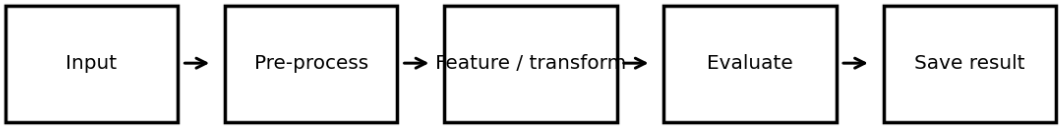


Figure 4.1 Processing workflow: Sharpening and Edge Preservation

Figure 4.1: Conceptual workflow for Task 4.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 4 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("crop.png",0).astype(np.float32)
blur=cv2.GaussianBlur(im,(5,5),1.0)
unsharp=np.clip(im+1.2*(im-blur),0,255).astype("uint8")
lap=cv2.Laplacian(im,cv2.CV_32F,ksize=3)
lap_img=np.uint8(np.clip(im-lap,0,255))
high=np.uint8(np.clip(1.5*im-blur,0,255))
cv2.imwrite("unsharp.png",unsharp)
cv2.imwrite("laplacian.png",lap_img)
cv2.imwrite("highboost.png",high)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 4 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

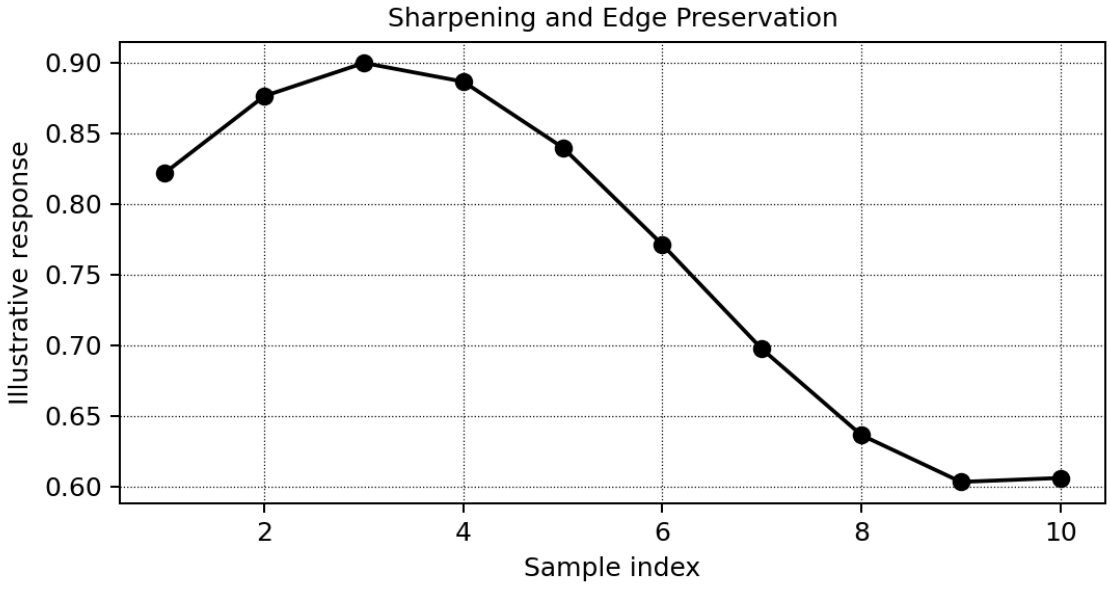


Figure 4.2: Conceptual response plot for Task 4. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied sharpening and edge preservation in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERS

1. Experiment Number and Title

Task 5: Frequency-Domain Low-Pass Filters.

2. Aim

Compute and plot the centred Fourier spectrum; implement ideal, Butterworth and Gaussian low-pass filters at two cutoffs; compare PSNR, SSIM, runtime, edge blur and ringing.

3. Objectives

- Understand the principle and purpose of frequency-domain low-pass filters.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

The present experiment is part of the design of a computer-vision solution for the stated application. The central engineering question is not merely whether an algorithm can produce an output, but whether the output is repeatable, measurable and suitable for the operating conditions. The experiment therefore separates input preparation, processing, evaluation and interpretation.

TASK 5 – DETAILED THEORY

6. Introduction

The Fourier transform represents an image as spatial frequency components. Low frequencies represent slowly varying intensity and illumination. High frequencies contain sharp edges, fine text strokes and some noise. Low-pass filtering suppresses high-frequency content, so it can reduce noise while softening character boundaries.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Page → FFT and centred magnitude spectrum → low-pass mask → inverse FFT → image/metric comparison
- Use identical cutoffs and image dimensions across filter families.
- Record Butterworth order and cutoff radius.

- Compare ringing and edge transition width, not only PSNR.

6.2 Mathematical Formulation

F(u,v)=FFT2(f).
Ideal LPF: H=1 for $D \leq D_0$, otherwise 0.
Butterworth: $H=1/[1+(D/D_0)^{\wedge}(2n)]$.
Gaussian: $H=\exp(-D^2/(2D_0^2))$.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 5 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

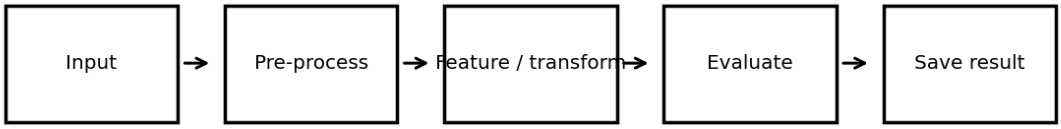


Figure 5.1 Processing workflow: Frequency-Domain Low-Pass Filters

Figure 5.1: Conceptual workflow for Task 5.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 5 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("page.png",0).astype(np.float32)
F=np.fft.fftshift(np.fft.fft2(im))
h,w=im.shape; y,x=np.ogrid[:h,:w]
D=np.sqrt((x-w/2)**2+(y-h/2)**2)
D0=40; order=2
masks={"ideal":(D<=D0).astype(float),
      "butterworth":1/(1+(D/(D0+1e-9))**(2*order)),
      "gaussian":np.exp(-(D**2)/(2*D0**2))}
for name,H in masks.items():
   out=np.abs(np.fft.ifft2(np.fft.ifftshift(F*H)))
   cv2.imwrite(name+"_lowpass.png",np.uint8(np.clip(out,0,255)))
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 5 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

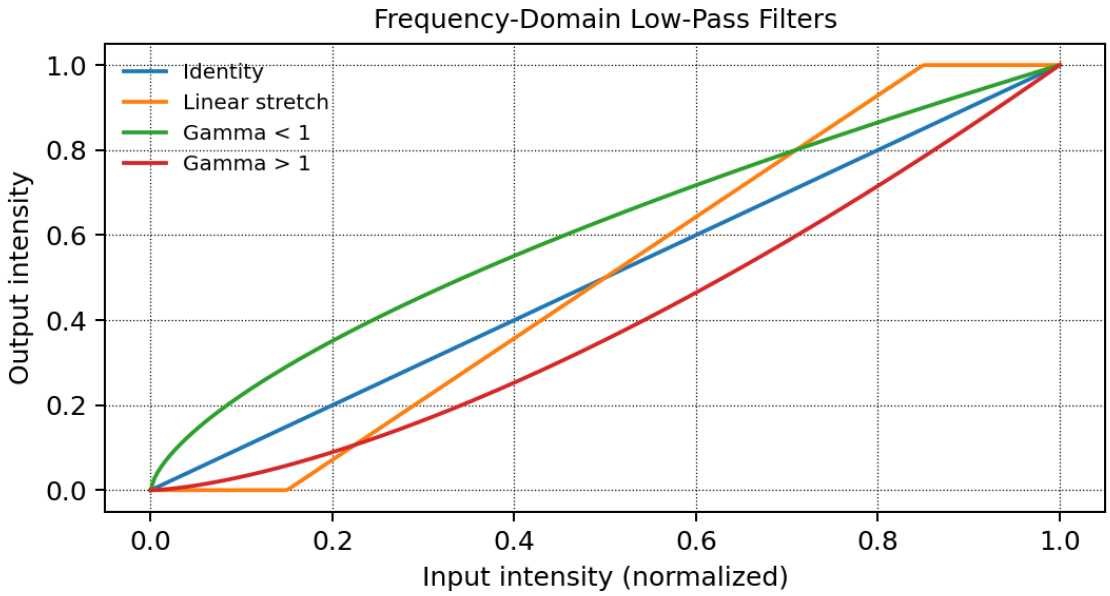


Figure 5.2: Conceptual response plot for Task 5. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied frequency-domain low-pass filters in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERS

1. Experiment Number and Title

Task 6: Frequency-Domain High-Pass Filters.

2. Aim

Implement ideal, Butterworth and Gaussian high-pass filters; compare edge contrast, background noise and text-region recall.

3. Objectives

- Understand the principle and purpose of frequency-domain high-pass filters.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

Input quality has a direct influence on the result. The image set should be documented with sample identifiers, capture conditions, resolution, colour format and annotation status. If multiple views belong to the same physical object or page, they should be assigned to the same data split. This avoids an overly optimistic evaluation caused by near-duplicate samples appearing in both training and test data.

TASK 6 – DETAILED THEORY

6. Introduction

High-pass filters attenuate slowly varying intensity and emphasize rapid changes. They can reveal text boundaries and fine details but may also amplify sensor noise, paper grain and compression artifacts. The filtered image must be normalized consistently before visual or quantitative comparison.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Page → FFT → high-pass mask → inverse FFT → edge contrast/background noise/text recall
- Construct high-pass masks as the complement of corresponding low-pass masks.
- Apply the same mask geometry and cutoff settings for a fair comparison.
- Check whether enhanced edges remain connected and readable.

6.2 Mathematical Formulation

$$HHP = 1 - HLP.$$

$$\text{Filtered image} = \text{real}(\text{IFFT2}(\text{FFT2}(\text{image}) \times HHP)).$$

$$\text{Edge contrast} = \text{mean}(\text{edge pixels}) - \text{mean}(\text{local background}).$$

$$\text{Recall} = TP/(TP+FN).$$

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 6 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

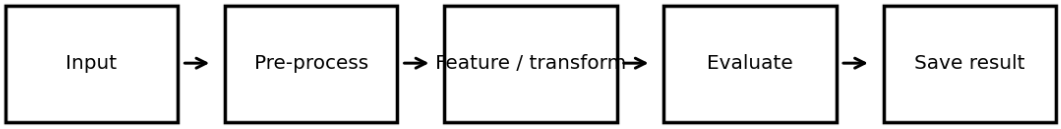


Figure 6.1 Processing workflow: Frequency-Domain High-Pass Filters

Figure 6.1: Conceptual workflow for Task 6.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 6 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("page.png",0).astype(np.float32)
F=np.fft.fftshift(np.fft.fft2(im))
h,w=im.shape; y,x=np.ogrid[:h,:w]
D=np.sqrt((x-w/2)**2+(y-h/2)**2); D0=35
lp=np.exp(-(D**2)/(2*D0**2))
filters={"ideal":1-(D<=D0),
        "gaussian":1-lp,
        "butterworth":1-1/(1+(D/D0)**4)}
for name,H in filters.items():
    out=np.real(np.fft.ifft2(np.fft.ifftshift(F*H)))
    out=cv2.normalize(out,None,0,255,cv2.NORM_MINMAX).astype("uint8")
    cv2.imwrite(name+"_highpass.png",out)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 6 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

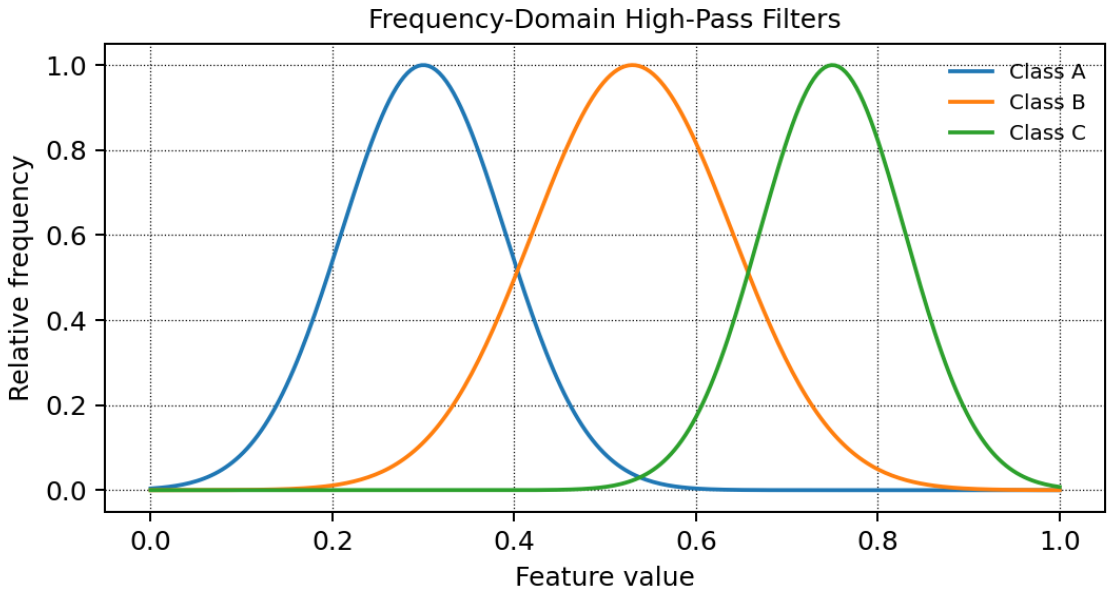


Figure 6.2: Conceptual response plot for Task 6. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied frequency-domain high-pass filters in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 7 – POINT, LINE AND EDGE DETECTION

1. Experiment Number and Title

Task 7: Point, Line and Edge Detection.

2. Aim

Implement point and line kernels, Sobel and Canny detectors; evaluate punctuation, table borders and diagrams using precision, recall and F1.

3. Objectives

- Understand the principle and purpose of point, line and edge detection.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

Parameters should be selected using the training or validation data. The final test set is reserved for one-time evaluation. Every experiment should record the parameter values, software versions, random seed and hardware details. These records allow another student or reviewer to repeat the work and investigate a result that differs from the expected behaviour.

TASK 7 – DETAILED THEORY

6. Introduction

Point detection identifies isolated intensity changes. Line kernels respond to structures with a preferred orientation. Sobel estimates image gradients in horizontal and vertical directions. Canny combines smoothing, gradient calculation, non-maximum suppression and hysteresis thresholding to produce thin connected edges.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Page → point/line kernels or gradient detector → response maps → annotated edge comparison
- Use horizontal and vertical line kernels for table structures.
- Tune Canny thresholds on validation images.
- Evaluate against annotated point, line and edge masks.

6.2 Mathematical Formulation

Gradient magnitude=$\sqrt{(G_x^2+G_y^2)}$.

Precision=TP/(TP+FP).

Recall=TP/(TP+FN).

F1=2×Precision×Recall/(Precision+Recall).

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 7 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

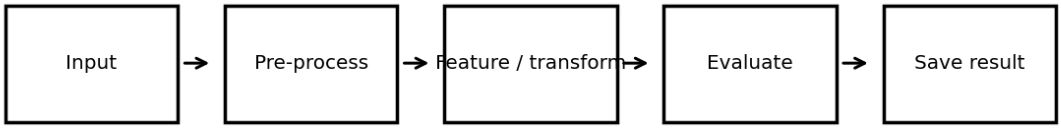


Figure 7.1 Processing workflow: Point, Line and Edge Detection

Figure 7.1: Conceptual workflow for Task 7.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 7 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("page.png",0)
point=np.array([[-1,-1,-1],[-1,8,-1],[-1,-1,-1]],np.float32)
horizontal=np.array([[-1,-1,-1],[2,2,2],[-1,-1,-1]],np.float32)
p=cv2.filter2D(im,-1,point)
line=cv2.filter2D(im,-1,horizontal)
gx=cv2.Sobel(im,cv2.CV_32F,1,0,ksize=3)
gy=cv2.Sobel(im,cv2.CV_32F,0,1,ksize=3)
sobel=cv2.convertScaleAbs(cv2.magnitude(gx,gy))
canny=cv2.Canny(im,60,150)
for name,a in [("point",p),("line",line),("sobel",sobel),("canny",canny)]:
    cv2.imwrite(name+".png",a)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 7 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

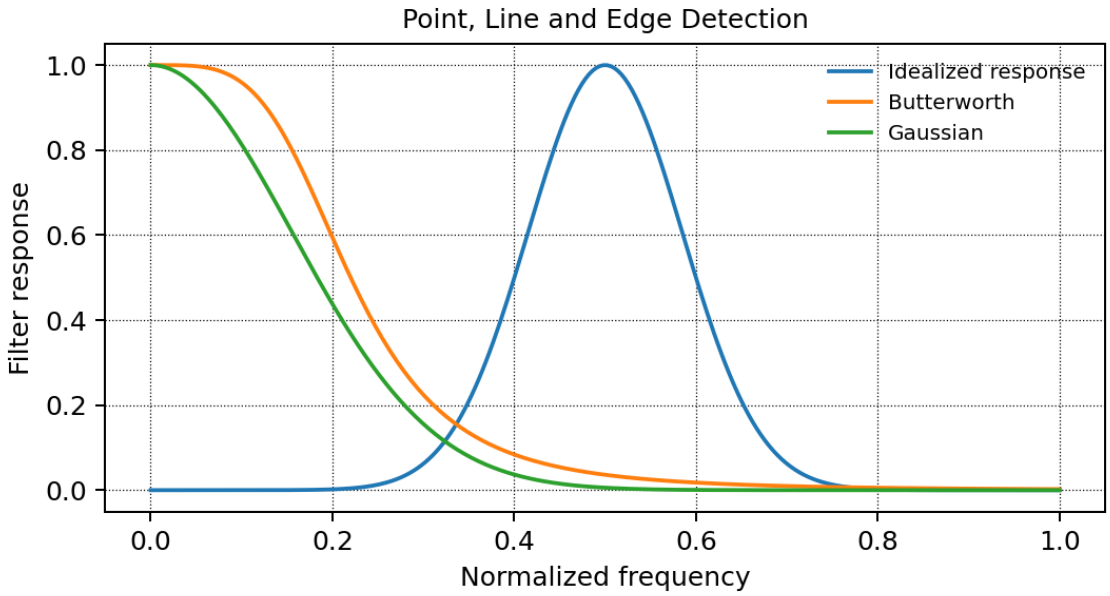


Figure 7.2: Conceptual response plot for Task 7. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied point, line and edge detection in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 8 – THRESHOLDING AND BINARY SEGMENTATION

1. Experiment Number and Title

Task 8: Thresholding and Binary Segmentation.

2. Aim

Compare fixed, Otsu and adaptive Gaussian thresholding on shadowed and evenly lit pages using Dice and IoU.

3. Objectives

- Understand the principle and purpose of thresholding and binary segmentation.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

Visual inspection is necessary but insufficient. A processed image may appear clearer while losing small defects or text strokes. Quantitative measures must therefore be interpreted alongside representative crops, error cases and the original annotations. The report should state the metric definition and the exact population over which it was calculated.

TASK 8 – DETAILED THEORY

6. Introduction

Thresholding separates foreground text from the page background. Fixed thresholding applies one global cutoff. Otsu selects a global threshold by maximizing between-class variance. Adaptive Gaussian thresholding estimates a local threshold from a neighbourhood and is useful when illumination changes across the page.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Page intensity → fixed/Otsu/adaptive threshold → binary mask → ground-truth Dice/IoU
- Use the same polarity convention for all masks.
- Record Otsu's selected threshold and adaptive block size/constant.
- Compare shadowed and evenly illuminated subsets separately.

6.2 Mathematical Formulation

Dice=2|A∩B|/(|A|+|B|).

IoU=|A∩B|/|A∪B|.

Otsu selects t that maximizes between-class variance.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 8 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

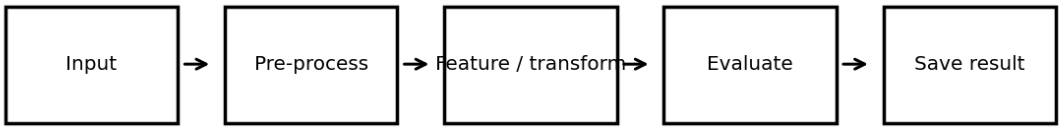


Figure 8.1 Processing workflow: Thresholding and Binary Segmentation

Figure 8.1: Conceptual workflow for Task 8.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 8 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("page.png",0)
_,fixed=cv2.threshold(im,150,255,cv2.THRESH_BINARY_INV)
t,otsu=cv2.threshold(im,0,255,cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
adaptive=cv2.adaptiveThreshold(im,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                    cv2.THRESH_BINARY_INV,31,10)
print("Otsu threshold",t)
cv2.imwrite("fixed.png",fixed)
cv2.imwrite("otsu.png",otsu)
cv2.imwrite("adaptive.png",adaptive)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 8 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

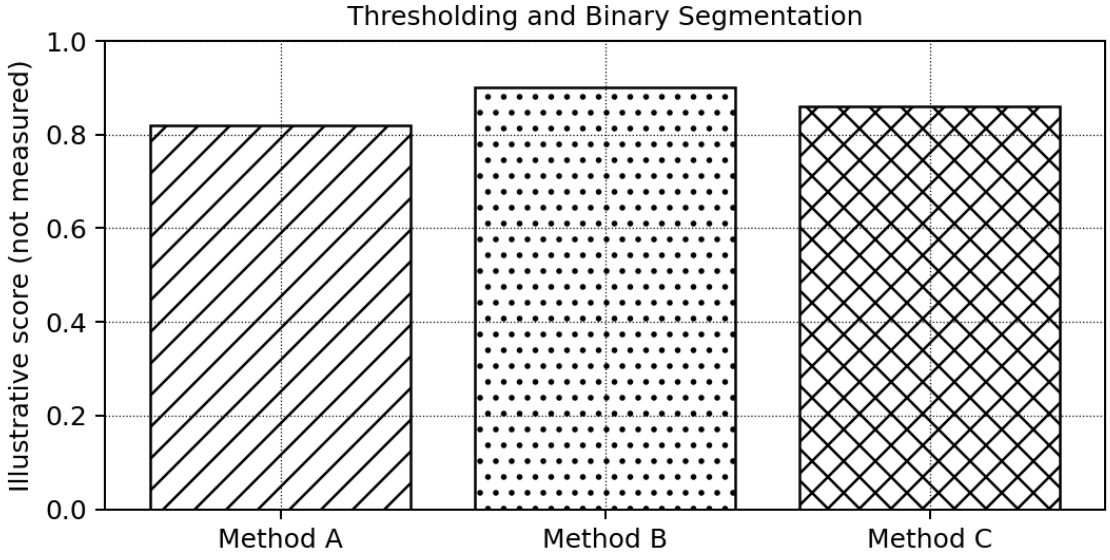


Figure 8.2: Conceptual response plot for Task 8. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied thresholding and binary segmentation in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 9 – REGIONS, BOUNDARIES AND WATERSHED

1. Experiment Number and Title

Task 9: Regions, Boundaries and Watershed.

2. Aim

Use connected components, watershed and morphology to segment text blocks, diagrams and page regions; evaluate IoU and over-/under-segmentation.

3. Objectives

- Understand the principle and purpose of regions, boundaries and watershed.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

The selected technique should be judged against the constraints of the scenario. Relevant considerations include accuracy, sensitivity to lighting and viewpoint, computational time, memory use, ease of implementation and failure handling. A method that performs well on clean examples may be unsuitable when glare, blur, occlusion or uneven illumination is present.

TASK 9 – DETAILED THEORY

6. Introduction

Connected components label connected foreground regions. Morphological opening removes small foreground structures, while closing bridges small gaps. Watershed uses marker regions and image gradients to separate touching objects or regions. The selected method must preserve meaningful text and diagram regions without merging unrelated blocks.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Binary foreground → morphology → connected components / watershed → region labels → IoU
- Record connectivity, structuring-element shape and size.
- Record watershed marker-generation method.
- Count split and merge errors relative to annotated regions.

6.2 Mathematical Formulation

Closing: $A \bullet B = (A \oplus B) \ominus B$.

IoU=intersection area/union area.

Over-segmentation: one reference region split into multiple predictions.

Under-segmentation: multiple reference regions merged into one prediction.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 9 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

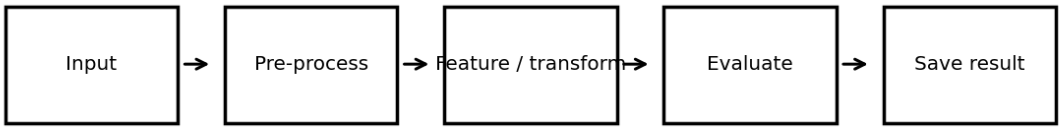


Figure 9.1 Processing workflow: Regions, Boundaries and Watershed

Figure 9.1: Conceptual workflow for Task 9.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 9 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("page.png",0)
_,bw=cv2.threshold(im,0,255,cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel=cv2.getStructuringElement(cv2.MORPH_RECT,(3,3))
closed=cv2.morphologyEx(bw,cv2.MORPH_CLOSE,kernel,iterations=1)
n,labels,stats,centroids=cv2.connectedComponentsWithStats(closed)
dist=cv2.distanceTransform(closed,cv2.DIST_L2,5)
_,sure=cv2.threshold(dist,.4*dist.max(),255,0); sure=np.uint8(sure)
unknown=cv2.subtract(closed,sure)
_,markers=cv2.connectedComponents(sure); markers+=1; markers[unknown==255]=0
markers=cv2.watershed(cv2.cvtColor(im,cv2.COLOR_GRAY2BGR),markers)
print("components",n-1,"watershed labels",len(np.unique(markers)))
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 9 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

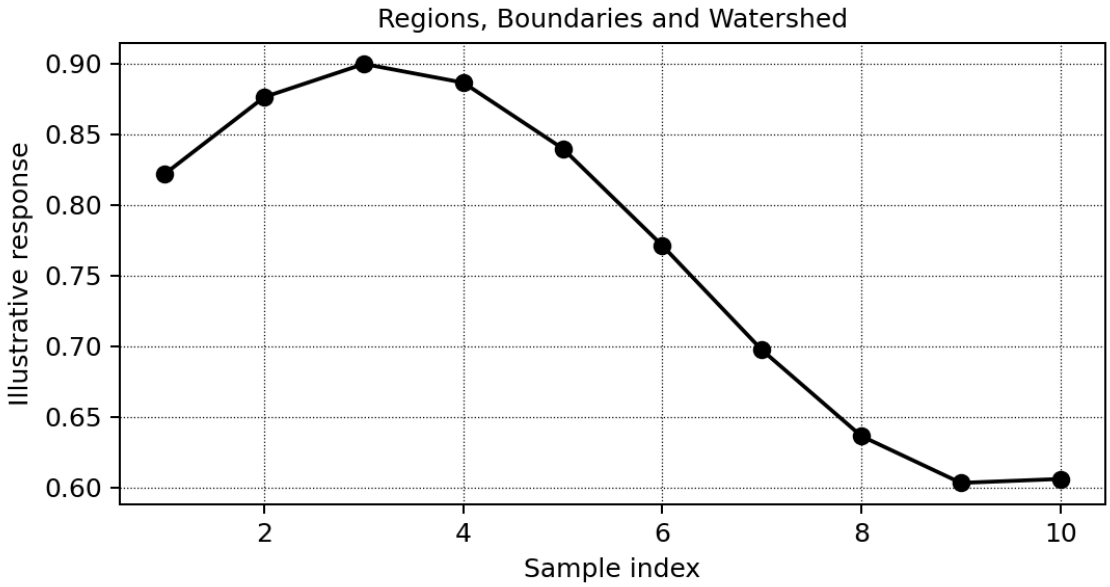


Figure 9.2: Conceptual response plot for Task 9. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied regions, boundaries and watershed in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 10 – INTEGRATED SCANNER AND ROBUSTNESS

1. Experiment Number and Title

Task 10: Integrated Scanner and Robustness.

2. Aim

Integrate enhancement, deskewing, page cropping and segmentation; evaluate a fixed test split and test robustness to shadow and blur.

3. Objectives

- Understand the principle and purpose of integrated scanner and robustness.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

The college library is digitizing forms photographed on phones. Pages may be shadowed, skewed, low contrast, stained or surrounded by clutter. The scanner must deskew each page, isolate text and diagrams, and preserve small characters for OCR. The assessment requires at least 30 page images with annotated page or text masks and a fixed test split.

The present experiment is part of the design of a computer-vision solution for the stated application. The central engineering question is not merely whether an algorithm can produce an output, but whether the output is repeatable, measurable and suitable for the operating conditions. The experiment therefore separates input preparation, processing, evaluation and interpretation.

TASK 10 – DETAILED THEORY

6. Introduction

The integrated scanner combines photometric enhancement with geometric correction and segmentation. Deskewing corrects page orientation, cropping removes surrounding clutter, and text segmentation creates a binary representation for downstream OCR. Robustness evaluation applies controlled shadow and blur changes to held-out pages.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Phone image → page boundary → perspective correction/deskew → enhancement → text/diagram segmentation → saved scan
- Keep train/validation/test splits fixed and page-independent.
- Measure segmentation quality and runtime for every page.
- Flag low-confidence or severely degraded pages for manual review.

6.2 Mathematical Formulation

IoU=|prediction∩truth|/|prediction∪truth|.

Runtime=end_time−start_time.

Robustness drop=baseline score−shifted-condition score.

Deskew angle may be estimated from page contours or dominant Hough lines.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 10 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

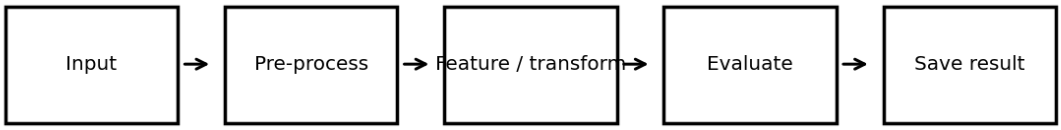


Figure 10.1 Processing workflow: Integrated Scanner and Robustness

Figure 10.1: Conceptual workflow for Task 10.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 10 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np, time
def scan_page(path):
    start=time.perf_counter()
    image=cv2.imread(str(path))
    if image is None: raise ValueError(path)
    gray=cv2.cvtColor(image,cv2.COLOR_BGR2GRAY)
    enhanced=cv2.createCLAHE(2.0,(8,8)).apply(gray)
    smooth=cv2.GaussianBlur(enhanced,(5,5),0)
    edges=cv2.Canny(smooth,50,150)
    contours,_=cv2.findContours(edges,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
    page=enhanced
    if contours:
        x,y,w,h=cv2.boundingRect(max(contours,key=cv2.contourArea))
        page=enhanced[y:y+h,x:x+w]
    mask=cv2.adaptiveThreshold(page,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY_INV,31,10)
    return page,mask,time.perf_counter()-start
page,mask,latency=scan_page("document.jpg")
cv2.imwrite("scanned_page.png",page)
cv2.imwrite("text_mask.png",mask)
print("latency",latency)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 10 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

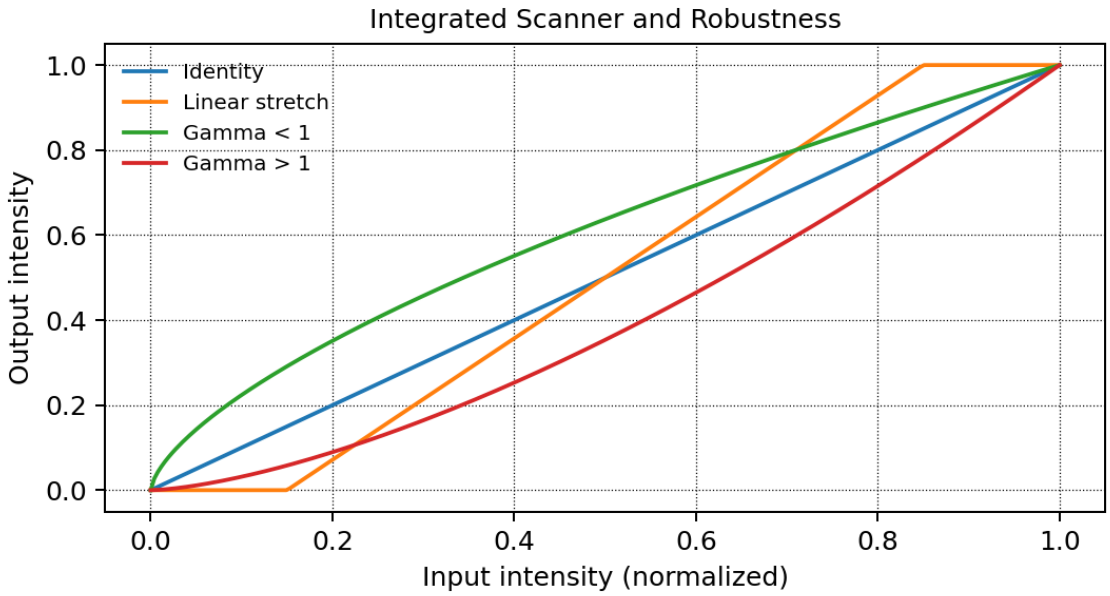


Figure 10.2: Conceptual response plot for Task 10. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied integrated scanner and robustness in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.